

PRODUCT · IPAD · ON-DEVICE ANALYSIS · ML/DATA

SentiVision

색으로 읽고, 작품으로 남기는 감정 전시

SentiVision은 사용자가 iPad에서 자유롭게 그림을 그리고, 작품의 색을 바탕으로 감정의 결을 해석해 전시 카드로 보관하는 감성 컴퓨팅 프로젝트입니다.

역할 제품 기획 · iPad 앱 · 분석 프로토타입 · 데이터 검증 · DevOps

현재 단계 iPad 기능 프로토타입

저장소 github.com/acertainromance401/SentiVision

최종 검증 2026-09-02

01. 프로젝트 한눈에 보기

말로 설명하기 어려운 감정을 드로잉의 흐름 안에서 돌아본다

텍스트나 음성 기반 감정 기록은 사용자가 먼저 감정을 언어로 정리해야 합니다. SentiVision은 그 부담을 줄이기 위해 **그림 그리기 → 색상 분석 → 전시형 해석 → 기록**을 하나의 흐름으로 설계했습니다.

항목	내용
제품	iPad 우선 프리미엄 감정 전시 앱
핵심 사용자	창작 과정에서 감정과 색의 관계를 기록하고 싶은 개인 창작자
핵심 경험	PencilKit 드로잉, 온디바이스 색상 분석, 감정 전시, 3D 분포, 로컬 아카이브
표현 원칙	의료 진단이나 단정이 아닌, 작품 해설과 같은 성찰형 표현
제품 전략	제작은 유료 iPad 앱, 후속 감상 경험은 무료 iPhone 동반 앱으로 분리
개인정보 방향	현재 분석과 기록을 기기 안에서 처리하는 온디바이스 우선 구조

프로젝트 질문

색상-감정 데이터 연구를 실제 사용 가능한 iPad 경험으로 연결하면서도, 분석 결과를 정답처럼 단정하지 않으려면 어떻게 설계해야 하는가?

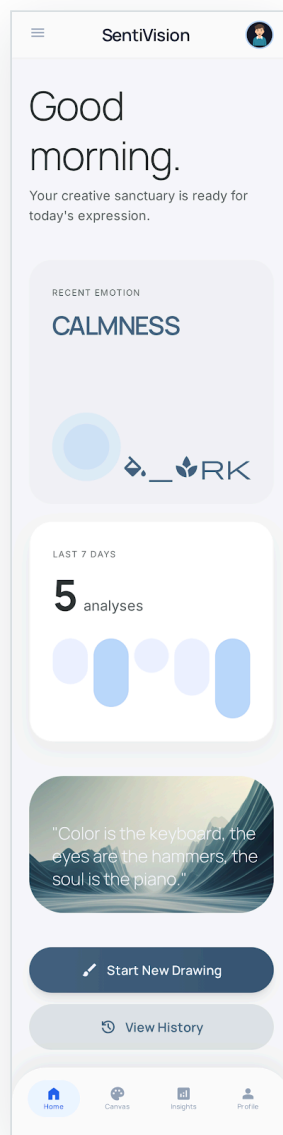
담당 범위

- PRD, 사용자 여정, 화면 흐름, WBS와 구현 우선순위 정리
- SwiftUI/PencilKit/SceneKit 기반 iPad 데모 구현
- Swift 온디바이스 색상·감정 분석과 로컬 아카이브 연결
- Python KNN/RandomForest 비교 및 데이터 보정 파이프라인 운영
- Node API, 테스트, 관측성, 기능 플래그, A/B, Canary와 CI/CD 자산 구축

02. 제품 경험

드로잉을 중심에 둔 일곱 단계

1. **개인 기준 설정**: 프로필명, 기준 감정, 기준 색상, 체감 강도를 저장합니다.
2. **자유 드로잉**: PencilKit 캔버스에서 색과 펜 굵기를 골라 그림을 그립니다.
3. **로컬 분석**: 캔버스의 전경 픽셀을 추출하고 최대 3개의 대표색을 계산합니다.
4. **감정 매핑**: 번들 CSV의 색상-감정 표본과 비교해 대표 감정과 점수를 만듭니다.
5. **감정 패밀리**: 개별 감정을 고요, 활력, 신비, 권위, 긴장, 연결, 온기, 회복, 집중, 그늘의 10개 결로 집계합니다.
6. **전시형 결과**: 해석문, 신뢰도, 대표색, 점수와 3D RGB 분포를 함께 보여줍니다.
7. **로컬 아카이브**: 결과를 전시 카드로 저장하고 다시 열람합니다.



제품 시안. 현재 데모는 별도 홈 대신 프로필 배너와 Analysis · Canvas · Archive 탭을 사용합니다.

03. 구현 구조

하나의 제품, 세 개의 목적별 트랙

트랙	현재 역할	핵심 기술	상태
iPad 앱	실제 사용자 경험과 주 분석 경로	SwiftUI, PencilKit, SceneKit, UIKit	기능 프로토타입
Node API	배포, 관측성, 계약 실험	Node.js HTTP, Jest, Prometheus	독립 프로토타입
Python	모델·데이터·추출 방식 검증	OpenCV, scikit-learn, pandas, matplotlib	연구/회귀 트랙

iPad 제품 경로

PencilKit 드로잉 → 전경 픽셀 추출 → K-means 대표색 → 최근접 감정 표본 → 감정 패밀리 집계 → 결과 전시 → UserDefaults 아카이브

Python 검증 경로

입력 이미지 → Saliency → K-means → KNN/RandomForest 비교 → 시각화 → 사용자 정정 → CSV 버전 관리

Node 운영 경로

팔레트 JSON → warmth/saturation/brightness 휴리스틱 → 감정 응답 → 구조화 로그/Prometheus 메트릭

왜 분리했는가

초기 문서는 서버 분석을 전제로 했지만 iPad 데모를 구현하면서 즉각적인 반응, 오프라인 동작, 개인정보 최소 전송이 더 중요한 제품 가치라고 판단했습니다. 따라서 앱의 주 경로는 온디바이스로 전환하고, Node API는 배포·관측성 실험, Python은 모델 검증에 집중시켰습니다.

이 분리는 아직 최종 아키텍처 결정이 아닙니다. 다음 단계에서 개인화 학습과 동기화 요구를 검증한 후 온디바이스 유지, 모델 번들 교체, 원격 추론 중 하나를 선택할 수 있도록 문서에 경계를 명시했습니다.

04. iPad 구현

코드로 연결된 핵심 기능

첫 실행과 개인 기준

`InitialSetupView`가 프로필명, 기준 감정, 기준 색상, 체감 강도를 받고 `AppStorage`에 저장합니다. 현재 기준 감정은 결과 비교 문장에 사용되며, 체감 강도의 모델 가중치 반영은 후속 범위입니다.

PencilKit 캔버스

`DrawingCanvasView`는 `PKCanvasView`를 SwiftUI에 연결합니다. 펜·벡터 지우개, 기본 색상 팔레트, 2~24pt 굵기, 전체 지우기를 제공하며 드로잉 변경을 `PKDrawing`으로 동기화합니다.

온디바이스 분석

`LocalEmotionAnalysisService`는 드로잉 이미지를 렌더링한 뒤 투명·배경 픽셀을 제외하고 최대 3개의 대표색을 K-means로 찾습니다. 각 중심색을 번들 표본과 RGB 거리로 비교하고, 픽셀 수와 예측 신뢰도로 감정 점수를 집계합니다.

감정 분포와 아카이브

`EmotionDistributionSceneView`는 최대 120개 표본을 SceneKit RGB 공간에 점으로 렌더링합니다.

`EmotionArchiveStore`는 분석 결과를 JSON으로 인코딩해 `UserDefaults`에 저장하며, 사용자는 카드 목록에서 과거 결과를 선택해 다시 볼 수 있습니다.

구현 파일	책임
<code>CanvasRootView.swift</code>	탭, 프로필, 캔버스 제어, 분석과 저장 흐름
<code>DrawingCanvasView.swift</code>	PencilKit 브리지
<code>LocalEmotionAnalysisService.swift</code>	대표색·감정·패밀리 분석
<code>EmotionDistributionSceneView.swift</code>	결과 카드와 3D RGB 분포
<code>EmotionArchiveView.swift</code>	로컬 전시 카드 목록과 재열람

05. 데이터 설계와 계보

어떤 데이터를 사용했는가

색상과 감정의 관계를 표현하는 기본 단위는 **RGB 좌표 1개와 감정 라벨 1개**입니다. 원천 데이터는 Sophi Grace의 Kaggle 데이터셋 **Color associations in Art: Pt 1 Warmth**이며 MIT 라이선스 고지와 함께 프로젝트용 파생 CSV로 관리합니다.

데이터 파일	규모	역할
color_emotion_labeled_updated.csv	113행	편집·정규화된 원본 기준선
color_emotion_labeled_augmented.csv	249행, 83개 감정	앱 번들과 Python 학습·검증의 현재 기준
emotion_family_classification.csv	107개 연결	83개 세부 감정을 10개 감정 패밀리로 연결
colorassociations_warmth - colorwarmth.csv	원천 자산	데이터 출처와 색상 온기 연구의 기준

색상-감정 CSV 스키마

필드	타입/범위	의미
emotion	문자열	CALMNESS, ENERGY 같은 감정 라벨
R, G, B	정수, 0~255	색상을 RGB 3차원 좌표로 표현한 특성
color_name	문자열/선택	사람이 읽는 색상 이름
color_label	문자열/선택	원천 데이터의 색상 식별 값

학습 특성은 R, G, B 세 개이고 예측 목표는 emotion 입니다. 라벨은 trim·대문자 통일을 적용하고 CORWARDICE → COWARDICE, LONLINESS → LONELINESS 처럼 확인된 오타자를 매핑합니다.

왜 보강 데이터가 필요한가

원본에는 한 감정당 표본이 하나뿐인 singleton이 많아 3-NN과 계층 분할 평가가 불안정했습니다. 이를 해결하기 위해 각 감정의 대표 RGB를 보존하면서 HSV 공간에서 작은 변형 2개만 추가하는 **하이브리드 C안**을 사용했습니다.

1. 각 감정의 첫 RGB를 대표점으로 유지합니다.
2. 감정 문자열에서 만든 결정적 seed로 hue를 최대 ± 0.006 , saturation/value를 약 2~3%만 이동합니다.
3. RGB 충돌 시 채널을 1~3만큼 미세 조정합니다.
4. 감정마다 대표점 1개와 변형점 2개를 만들어 총 249행, 감정당 3표본을 구성합니다.
5. (emotion, R, G, B) 중복과 동일 RGB의 감정 충돌을 검사합니다.

왜 대규모 합성이나 외부 데이터 결합 대신 작은 HSV 변형을 택했는가

이 단계의 목적은 데이터가 많아 보이게 만드는 것이 아니라 **기존 색상-감정 의미를 훼손하지 않으면서 파이프라인을 비교 가능한 상태로 만드는 것이었습니다**. 무작위 RGB 노이즈는 색조 자체를 바꿔 원래 감정 연상을 깨뜨릴 수 있고, 여러 외부 데이터셋을 바로 합치면 라벨 정의와 조사 맥락이 달라 출처 차이가 모델 성능 차이로 섞입니다. 반면 HSV의 작은 이동은 사람이 같은 계열로 인식할 가능성이 높은 범위에서 밝기와 채도 변화를 만들 수 있습니다.

결정적 seed를 사용한 이유도 재현성 때문입니다. 같은 입력에서 항상 같은 파생 표본이 생성되어야 모델 변경 전후의 차이를 코드 변화로 해석할 수 있습니다. 다만 이 선택은 **학습 안정화를 위한 공학적 보강일 뿐** 새로운 사용자 응답을 수집한 것이 아닙니다. 실제 정정 데이터가 감정별로 충분히 쌓이면 합성 표본의 비중을 낮추고, 출처별 holdout으로 일반화 성능을 다시 확인하는 것이 교체 조건입니다.

데이터 해석 원칙

이 데이터는 임상적 감정 정답이나 사용자 집단의 확률 분포가 아닙니다. 색상 연상 데이터에서 파생된 소규모 의미 매핑이므로, 제품에서는 진단이 아니라 작품을 돌아보기 위한 해석 재료로 사용합니다.

06. 온디바이스 분석 원리

1단계 · 드로잉을 픽셀 데이터로 바꾸기

PencilKit의 `PKDrawing` 을 `UIImage` 로 렌더링한 뒤 Core Graphics의 RGBA 버퍼로 읽습니다. 계산량을 줄이기 위해 가로·세로 2픽셀 간격으로 샘플링하고 다음 조건으로 투명 배경과 거의 흰 배경을 제외합니다.

$\alpha \geq 24$ 이고, $\max(R,G,B) < 245$ 또는 $\text{HSV saturation} > 0.08$ 인 픽셀을 전경으로 선택

필터 결과가 비어 있으면 alpha가 0보다 큰 픽셀을 다시 사용해 단색·저채도 그림에서도 분석이 중단되지 않게 합니다.

2단계 · K-means로 대표색 찾기

전경에 서로 다른 색이 충분하면 최대 3개 클러스터를 만듭니다. 각 픽셀을 가장 가까운 중심에 배정하고, 중심을 소속 픽셀의 RGB 평균으로 갱신하는 과정을 최대 8회 반복합니다. 중심이 더 이상 변하지 않으면 조기 종료합니다.

거리: $d^2(x, \mu) = (R_x - R_\mu)^2 + (G_x - G_\mu)^2 + (B_x - B_\mu)^2$

목적: $J = \sum_k \sum_{x \in C_k} ||x - \mu_k||^2$ 를 작게 만드는 대표 중심 μ_k 탐색

K-means는 감정을 분류하는 모델이 아니라 수많은 드로잉 픽셀을 **대표 RGB 최대 3개로 압축하는 비지도 학습 단계**입니다.

왜 평균색 하나나 모든 픽셀 대신 최대 3개의 K-means 중심을 사용했는가

평균색 하나는 서로 다른 색을 섞어 실제 그림에 없던 중간색을 만들 수 있습니다. 반대로 모든 픽셀을 바로 분류하면 같은 확의 반복 픽셀이 결과를 과도하게 지배하고 기기 연산량도 커집니다. 최대 3개의 중심은 여러 색으로 그린 작품의 구조를 남기면서도 이후 감정 매핑 횟수를 작게 유지하는 절충입니다. 클러스터 수를 고정된 큰 값으로 두지 않은 것은 단색 작품에 의미 없는 중심을 강제로 만들지 않기 위해서입니다.

Swift에서 반복 횟수를 8회로 제한하고 중심 변화 시 조기 종료한 것은 모바일 상호작용의 즉시성을 우선한 결정입니다. 이 값은 최적 군집을 보장하는 통계적 기준이 아니라 작은 캔버스 표본에서 지연을 제한하기 위한 제품 기준입니다. 향후 작품 유형별 대표색 재현율과 실기기 지연을 함께 측정해 3개보다 많은 중심이 해석 품질을 유의미하게 높일 때 조정할 수 있습니다.

3단계 · 3-NN으로 감정 매핑하기

각 대표색과 249개 색상 표본 사이의 RGB 제곱거리를 계산하고 가장 가까운 3개를 선택합니다. 감정 라벨 다수결로 승자를 정하며, 득표수가 같으면 해당 감정 이웃들의 평균 거리가 더 짧은 쪽을 선택합니다.

```
cluster confidence = 승리 감정의 이웃 수 / 3
```

따라서 가능한 기본 값은 1/3, 2/3, 1입니다.

Python의 `KNeighborsClassifier(n_neighbors=3)` 와 같은 원리를 Swift에서 직접 구현해 모델 서버나 별도 ML 런타임 없이 동작하게 했습니다.

왜 신경망이나 원격 모델 대신 3-NN을 제품 기준선으로 삼았는가

현재 데이터는 83개 감정에 감정당 3표본뿐이어서 학습 파라미터가 많은 모델은 데이터의 의미보다 파생 노이즈를 외울 위험이 큼니다. 3-NN은 “색 공간에서 가까운 표본은 비슷한 감정 연상을 가진다”는 프로젝트의 가정을 거리와 이웃으로 그대로 보여줍니다. 결과가 이상할 때 어떤 RGB 표본이 투표했는지 추적할 수 있다는 점도 감정처럼 설명 책임이 필요한 기능에 중요했습니다.

`k=1` 은 하나의 표본 오류에 민감하고, 큰 `k` 는 83개 세부 라벨 사이의 국소 차이를 희석합니다. `k=3` 은 최소한의 다수결과 동률 처리 가능성을 제공하면서 감정당 3표본이라는 현재 데이터 구조와 맞습니다. 서버 추론을 제외한 이유는 캔버스 이미지를 전송하지 않아도 되어 개인정보·오프라인 사용·응답 지연을 한 번에 줄일 수 있기 때문입니다. 실제 사용자 데이터가 늘어나고 거리 기반 모델의 오류가 반복되면 CIELAB· ΔE , 거리 가중 KNN, 경량 Core ML 모델을 같은 고정 검증셋에서 비교하는 것이 다음 판단 단계입니다.

4단계 · 그림 전체 점수와 감정 패밀리 만들기

대표색마다 차지한 픽셀 수와 3-NN 득표 비율을 곱해 감정별 가중치를 만들고 전체 합으로 나눕니다.

$$W(e) = \Sigma(\text{cluster pixel count} \times \text{cluster confidence})$$
$$\text{score}(e) = W(e) / \Sigma W(\text{all emotions})$$

세부 감정은 10개 패밀리로 다시 집계합니다. primary 관계는 1.0, secondary 관계는 0.65를 곱하며 하나의 감정이 여러 패밀리에 기여할 수 있습니다.

$$\text{family score}(f) = \Sigma \text{score}(e) \times \text{membership weight}(e, f)$$

확률이 아니라 해석 점수

앱의 `confidence` 와 `family score` 는 softmax, 베이지안 추론, 확률 보정(calibration)으로 얻은 값이 아닙니다. 이웃 투표와 픽셀 비중을 정규화한 상대 점수이므로 “감정일 확률 80%”가 아니라 “현재 규칙에서 이 감정이 차지한 비중”으로 읽어야 합니다.

07. Python 머신러닝 검증

제품 코드와 연구 코드를 분리해 비교 가능성을 유지했다

원본 기준선은 `base_model/`, 개선·검증 코드는 `test/`에 분리했습니다. 앱은 빠른 로컬 추론에 집중하고 Python은 이미지 추출 방식, 분류기 선택, 데이터 변경의 영향을 반복 측정합니다.

이미지 전처리와 색상 추출

1. OpenCV로 이미지를 읽고 BGR에서 RGB로 변환합니다.
2. 분석 해상도를 100×100으로 줄여 연산량과 노이즈를 제어합니다.
3. 5×5 Gaussian Blur로 미세 노이즈를 줄입니다.
4. Laplacian으로 밝기 변화가 큰 경계를 구하고 threshold 10으로 현저성 마스크를 만듭니다.
5. `paint_region` 변형에서는 채도·밝기·경계 강도를 결합하고 percentile threshold로 실제 칠한 영역을 더 넓게 찾습니다.
6. scikit-learn K-means를 `random_state=42`, `n_init=10`, 최대 3클러스터로 실행합니다.

비교한 지도학습 모델

모델	설정	선택 이유	프로젝트에서의 역할
K-Nearest Neighbors	<code>k=3</code>	RGB 공간에서 “가까운 색은 비슷한 감정”이라는 가정을 직접 표현	해석 가능한 기준선이자 앱 구현 원형
Random Forest	100 trees, <code>random_state=42</code>	여러 결정 트리의 비선형 경계를 앙상블해 KNN과 다른 분류 특성 제공	대안 모델 성능 비교

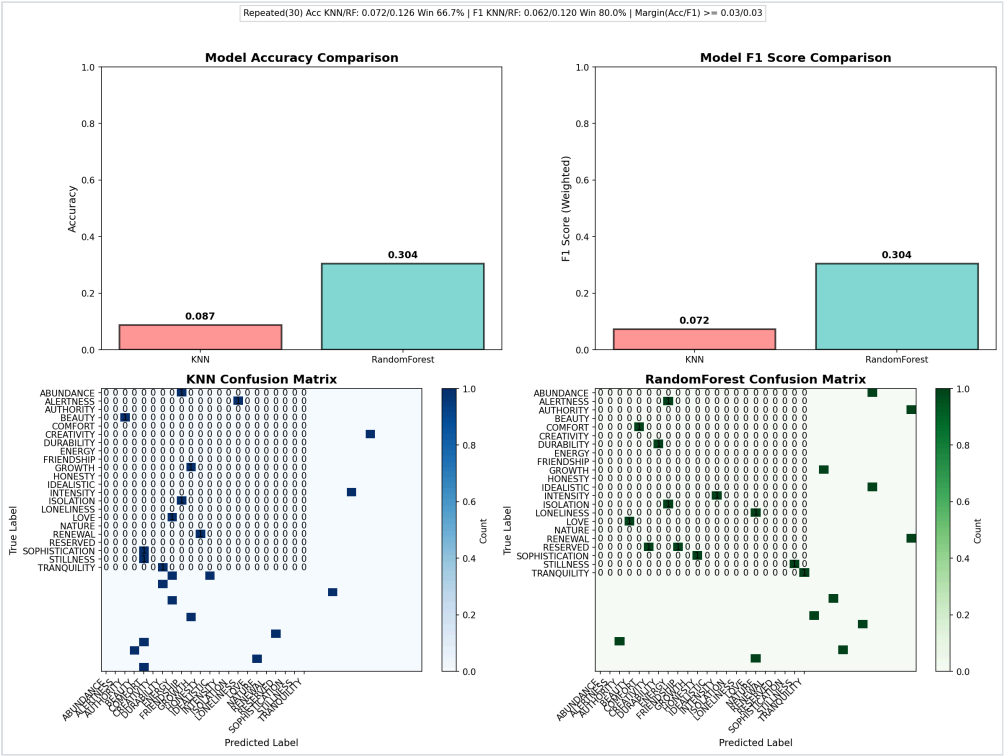
KNN은 별도 파라미터 학습보다 표본 자체를 기억하는 instance-based model입니다. RandomForest는 RGB 채널 분할 규칙을 서로 다른 bootstrap 트리에서 학습하고 다수결합니다. 데이터가 작기 때문에 복잡한 신경망보다 두 모델의 편향 차이를 비교하는 방식을 선택했습니다.

Random Forest를 함께 둔 의도는 더 복잡한 모델이 무조건 낫다는 것을 증명하는 데 있지 않습니다. KNN은 거리와 국소 이웃에 의존하지만 Random Forest는 RGB 축의 비선형 분할을 학습하므로, 두 모델의 결과 차이는 현재 데이터가 **거리 기반 구조인지, 채널 임계값 기반 구조인지**를 탐색하는 단서가 됩니다. 100개 트리는 실행 비용을 과도하게 키우지 않으면서 단일 트리의 분할 변동을 평균화하기 위한 비교 기준이고, `random_state=42`는 실험 재현성을 위한 고정값입니다.

Random Forest가 한 번 높은 점수를 냈다는 이유만으로 앱 모델을 교체하지 않았습니다. 파생 표본 내부 분할에서는 같은 원본에서 나온 변형이 학습·검증에 나뉘어 성능이 낙관적으로 보일 수 있기 때문입니다. 그래서 Accuracy와 Weighted F1이 모두 평균 3%p 이상 우세하고 반복 분할에서도 승률이 유지되는 경우만 후보 신호로 봅니다. 이후 실제 사용자 단위의 독립 검증에서도 우세하고 Core ML 변환 후 지연·용량 기준을 만족할 때 제품 적용을 검토합니다.

평가 설계

평가 장치	구현 목적
RGB 0~1 정규화	거리 기반 KNN의 특성 범위를 명확하게 유지
고정 검증셋 20%	데이터 변경 전후를 같은 표본에서 비교
최대 5-fold 교차검증	단일 분할 결과에 대한 의존도 완화
30회 repeated holdout	seed 42~71의 평균·표준편차와 모델 승률 확인
Accuracy	전체 예측 중 정답 비율
Weighted F1	클래스별 precision/recall을 표본 수로 가중해 불균형 반영
Confusion Matrix	어떤 감정끼리 혼동되는지 시각적으로 확인
우세 margin 0.03	Accuracy와 F1 평균이 모두 3%p 이상일 때만 RF 우세로 판정



저장소에 보존된 모델 비교 대시보드. 수치는 데이터 분할 조건에 종속되므로 제품 정확도 주장보다 모델 선택과 회귀 확인의 근거로 사용합니다.

모델 결과를 과장하지 않는 이유

- 249행에 83개 감정으로 감정당 3표본뿐이라 일반화 근거가 제한적입니다.
- 두 변형 표본이 같은 대표색에서 생성되어 통계적으로 독립적인 사람 응답이 아닙니다.
- RGB 유클리드 거리는 간단하지만 인간 지각에 균일한 CIELAB·ΔE 거리가 아닙니다.
- 현재 검증셋은 외부 사용자 집단의 독립 라벨이 아니라 같은 파생 데이터 내부 분할입니다.
- 따라서 대시보드는 제품 정확도 인증이 아니라 코드 회귀와 모델 후보 비교에 사용합니다.

다음 모델 단계에서는 실제 사용자 정정 데이터를 별도 검증셋으로 분리하고, CIELAB 색공간·거리 가중 KNN·확률 calibration을 비교해야 합니다.

08. 기술 스택과 각 도구의 책임

iPad 제품 계층

기술	사용 위치	선택 이유
SwiftUI	온보딩, 탭, 결과, 아카이브	상태 기반 UI와 iPad 레이아웃 구성
PencilKit	드로잉 캔버스와 펜/지우개	Apple Pencil 입력과 PKDrawing 데이터 제공
UIKit / Core Graphics	드로잉 이미지화와 RGBA 버퍼 접근	픽셀 단위 전경 추출과 UIColor 처리
SceneKit	RGB 감정 분포 3D 렌더링	색상 표본을 공간 좌표로 탐색
Foundation	CSV, Codable, UserDefaults	번들 데이터 로드와 로컬 아카이브 영속화

Python 데이터·ML 계층

라이브러리	책임
pandas	CSV 로드, 정규화 열 생성, 중복 제거, 피드백 병합, 이력 저장
NumPy	RGB 배열 연산, 평균·표준편차, percentile, 마스크 처리
OpenCV	이미지 로드·리사이즈, Gaussian Blur, Laplacian, threshold
scikit-learn	KMeans, KNeighborsClassifier, RandomForestClassifier, 데이터 분할과 평가 지표
matplotlib	RGB 3D 분포, Saliency, 대표색, 성능 대시보드 생성
pytest / Ruff	Python smoke test와 정적 품질 검사

Node 운영·실험 계층

Node API는 외부 ML 라이브러리 없이 표준 http 모듈로 구성했습니다. 입력 팔레트의 색상별 weight를 정규화한 뒤 warmth, saturation, brightness를 가중 합산합니다.

```
score = 0.45 × signedWarmth + 0.35 × saturation + 0.20 × (brightness - 0.5)
heuristic confidence = clamp(0.55 + 0.40 × |score|, 0.55, 0.99)
```

이 값 역시 학습된 확률이 아니라 API 계약과 관측성 검증을 위한 휴리스틱입니다. Jest로 파싱·응답·메트릭을 검증하고, SHA-256 해시로 같은 사용자가 항상 같은 A/B variant에 배정되게 했습니다. NDJSON 이벤트 로그, Prometheus 형식 메트릭, Docker·Grafana 구성을 통해 모델 외의 운영 경로도 함께 실험했습니다.

09. 기술 선택의 의사결정 기록

선택은 기술 선호가 아니라 제품 단계의 제약에서 출발했다

네이티브 iPad 프레임워크를 중심에 둔 이유

이 제품의 핵심 입력은 Apple Pencil 드로잉이고 결과 확인은 창작 흐름 안에서 즉시 일어나야 합니다. 웹 캔버스나 크로스플랫폼 프레임워크도 검토할 수 있지만, PencilKit은 압력·획·지우개와 PKDrawing 직렬화를 운영체제 수준에서 제공하고 SwiftUI는 온보딩·설정·아카이브의 상태 흐름을 같은 코드베이스에서 연결합니다. Core Graphics를 함께 사용한 것은 PencilKit이 입력에는 강하지만 픽셀 기반 분석 버퍼를 직접 제공하지 않기 때문입니다.

SceneKit은 색상 표본을 RGB 3축으로 보여주는 현재 요구를 적은 코드로 충족해 선택했습니다. 고급 렌더링이 목표였다면 Metal이 더 적합하지만, 현 단계에서 셰이더와 렌더 파이프라인을 직접 관리하는 비용은 사용자 가치보다 큼니다. 즉 네이티브 선택의 명분은 최고 성능 자체가 아니라 **입력 정확도, 오프라인 처리, 구현 범위의 통제**입니다. 분석 규모가 커져 프레임 저하가 측정되거나 표현 요구가 SceneKit의 한계를 넘을 때 Metal 또는 다른 렌더러를 검토합니다.

온디바이스 제품과 Python 연구 환경을 분리한 이유

Python은 pandas·OpenCV·scikit-learn으로 데이터와 모델 가설을 빠르게 바꾸고 시각화하기 좋지만, 그대로 iPad 제품 런타임에 넣으면 배포 크기와 의존성 관리가 커집니다. 반대로 Swift만으로 모든 실험을 수행하면 교차검증, 모델 비교, 통계 요약의 다시 구현해야 해 연구 속도가 떨어집니다. 그래서 Python은 **증거를 만드는 실험실**, Swift는 **검증된 규칙을 사용자 경험으로 전달하는 런타임**으로 책임을 나눴습니다.

두 구현이 달라지는 위험은 고정 CSV, 동일한 $k=3$, 대표색 최대 3개, 재현 가능한 seed처럼 비교 가능한 계약으로 줄였습니다. 이후 모델이 단순 규칙으로 표현하기 어려워지면 Python 학습 결과를 Core ML 번들로 내보내되, 캔버스 흐름과 개인정보 요구는 계속 앱이 소유하도록 설계합니다.

Node 프로토타입을 표준 모듈과 휴리스틱으로 시작한 이유

Node 트랙의 질문은 “최고 정확도의 감정 모델을 서버에 만들 수 있는가”가 아니라 “API 계약, 기능 플러그, 관측성, A/B 배정, 롤백을 실제로 운영할 수 있는가”였습니다. 이 질문을 검증하는 데 무거운 ML 프레임워크를 먼저 붙이면 모델 오류와 운영 오류를 구분하기 어렵습니다. 표준 http 모듈과 결정적 휴리스틱을 사용해 응답과 메트릭을 예측 가능하게 만들고 운영 경로를 먼저 검증했습니다.

SHA-256 기반 배정은 같은 사용자가 요청할 때마다 실험군이 바뀌지 않으면서 별도 상태 저장소가 필요 없다는 이유로 선택했습니다. NDJSON은 초기 단계에서 한 줄 단위로 기록·복구하기 쉽고, Prometheus 형식은 시계열 지표 도구와 연결하기 쉽습니다. 다만 트래픽과 동시 쓰기가 늘면 NDJSON은 데이터베이스나 이벤트 스트림으로, 표준 HTTP 라우팅은 검증·인증·스키마 도구를 갖춘 프레임워크로 교체해야 합니다.

의사결정의 공통 기준

현재 선택은 가장 화려한 기술보다 작은 데이터, iPad 우선 경험, 개인정보, 오프라인 동작, 설명 가능성, 재현성이라는 제약을 우선한 결과입니다. 각 선택에는 한계와 교체 조건을 함께 기록해, 데이터와 사용자 증거가 바뀌면 기술도 바꿀 수 있도록 했습니다.

10. 엔지니어링 품질

기능 구현 밖의 운영 가능성도 함께 검증했다

테스트

- Jest: 팔레트 파싱, 휴리스틱 감정 분석, API health/analyze/metrics, 기능 플래그, A/B 할당
- Playwright: 정적 프론트 프로토타입 렌더링
- Python: smoke test, 고정 검증셋 기반 모델 비교와 분석 스크립트
- iPad: Xcode 빌드와 실기기 중심 수동 흐름 검증

배포와 관측성

- Docker 이미지와 Docker Compose 기반 Node API · Prometheus · Grafana 구성
- `/health`, `/metrics`, 구조화 요청 로그
- GitHub Actions 기반 테스트, 보안 스캔, Docker, Pages, AWS ECS 배포 정의
- DORA 지표 수집, Canary 단계·오류율·지연 기준과 롤백 시뮬레이터

실험 기반

- 사용자 ID 해시 기반의 안정적인 A/B variant 할당
- 기능별 퍼센티지 롤아웃과 대상 사용자 override
- NDJSON 이벤트 기록과 2주 A/B 분석 자산

중요한 경계

CI/CD와 AWS 정의가 존재한다는 것은 운영 배포 완료란 뜻하지 않습니다. 현재 포트폴리오는 코드와 설정으로 검증 가능한 준비 상태를 설명하며, 실제 사용자 출시와 운영 KPI는 후속 단계입니다.

11. 진행 과정과 의사결정

Baseline에서 제품 경험까지

단계	진행 내용	결과
1. 문제·데이터 정의	색상-감정 CSV와 Python 기준선 정리	분석 가능성 및 데이터 이슈 확인
2. 모델 비교	KNN/RandomForest, 고정 검증셋, 시각화	모델 변경을 수치와 산출물로 비교
3. 제품 설계	PRD, 사용자 여정, 와이어프레임, WBS	전시형 해석과 iPad 우선 방향 확정
4. 운영 프로토타입	Node API, 테스트, Docker, 메트릭, 실험	배포·관측·롤백 요구 검증
5. iPad 구현	온보딩, 캔버스, 로컬 분석, 3D 분포, 아카이브	핵심 사용자 흐름을 기기 안에서 연결
6. 감정 패밀리	10개 패밀리 데이터와 결과 집계	개별 라벨을 더 읽기 쉬운 감정 결로 구조화
7. 문서 재정렬	앱·API·Python의 역할과 상태 최신화	구현과 계획의 불일치 제거

배운 점

- 연구용 정확도와 제품의 해석 품질은 같은 지표가 아닙니다. 결과 문장, 불확실성 표현, 사용자 정정 경로가 함께 설계되어야 합니다.
- 서버 우선 설계가 항상 제품에 최적인 것은 아닙니다. iPad에서는 지연, 오프라인, 개인정보 요구 때문에 온디바이스 경로가 더 자연스러웠습니다.
- 감정 라벨 수가 많을수록 UI 이해도가 자동으로 높아지지 않습니다. 감정 패밀리는 세밀한 모델 출력과 읽기 쉬운 제품 언어 사이의 번역 계층입니다.
- 문서는 목표와 현재 상태를 분리해야 합니다. 요구사항을 지우지 않으면서 구현·프로토타입·백로그를 명시하는 방식으로 정리했습니다.

12. 현재 상태와 다음 단계

구현 상태

구분	범위
구현	iPad 온보딩, PencilKit 캔버스, 로컬 대표색·감정 분석, 감정 패밀리, 결과 전시, 3D RGB 분포, 로컬 아카이브
프로토타입	Node 팔레트 분석 API와 관측성, Python 모델·데이터 검증, 실험·배포 자동화 자산
백로그	감정 수정·메모, 개인 분포 학습 반영, 고급 색상 입력, 작품 이미지 아카이브, 원격 동기화·인증, iPhone 감상 앱

우선순위

- 1. **개인화 루프 완성**: 정정 감정과 메모를 저장하고 개인 분포에 반영합니다.
- 2. **창작 도구 확장**: 색상 휠, HEX/RGB, 스포이트, 재사용 팔레트를 추가합니다.
- 3. **앱 품질 자동화**: Swift 단위 테스트와 실기기 회귀 체크리스트를 구축합니다.
- 4. **분석 경계 결정**: 온디바이스 유지, 교체 가능한 모델 번들, 원격 추론의 비용과 개인정보를 비교합니다.
- 5. **감상 앱 검증**: 유료 iPad 경험이 안정된 뒤 무료 iPhone 동반 앱의 전환 효과를 검증합니다.

SentiVision은 분석 모델만 만든 프로젝트가 아닙니다.

색상 데이터 연구를 iPad의 창작 경험, 전시형 결과 언어, 로컬 기록, 운영 검증 체계로 연결하며 아이디어가 실제 제품 흐름이 되는 과정 전체를 다뤘습니다.

Repository
<https://github.com/acertainromance401/SentiVision>